MCP vs API

Model Context Protocol explained, and how it differs from traditional APIs

What is MCP?

An open standard

Introduced by Anthropic in late 2024, MCP defines how AI applications connect to external tools, data sources and services.

Think of it as USB-C for AI: build one MCP server per tool, and any MCP-compatible app can use it.

Servers expose tools, resources and prompts over JSON-RPC 2.0.

Before MCP

$N \times M$

custom integrations for N AI apps and M tools

With MCP

$N + M$

one server per tool, one client per app

Architecture: Host, Client, Server

01 Host

The AI application the user works in.

Claude Desktop, Claude Code, Cursor, or your own agent app.

02 Client

Lives inside the host and keeps a 1:1 connection with one MCP server.

One client per server.

03 Server

A lightweight program exposing tools, data and prompts from a system.

Usually wraps an API, database or SaaS app.

Transport: JSON-RPC 2.0 over stdio (local subprocess) or Streamable HTTP (remote, OAuth)

What a Server Exposes

Primitive	Controlled by	Purpose	Example
Tools	Model	Actions the LLM can call	create_ticket, run_sql
Resources	Application	Read-only data and context	files, DB rows, docs
Prompts	User	Reusable prompt templates	a code-review workflow

Client-side features: Sampling (server asks the host LLM for a completion), Roots, and Elicitation (server asks the user for input)

The Request Flow

1

Initialize

Handshake and capability negotiation

2

Discover

tools/list returns names, descriptions and schemas

3

Decide

The LLM sees the schemas and picks a tool

4

Execute

Host sends tools/call and the server runs it

5

Respond

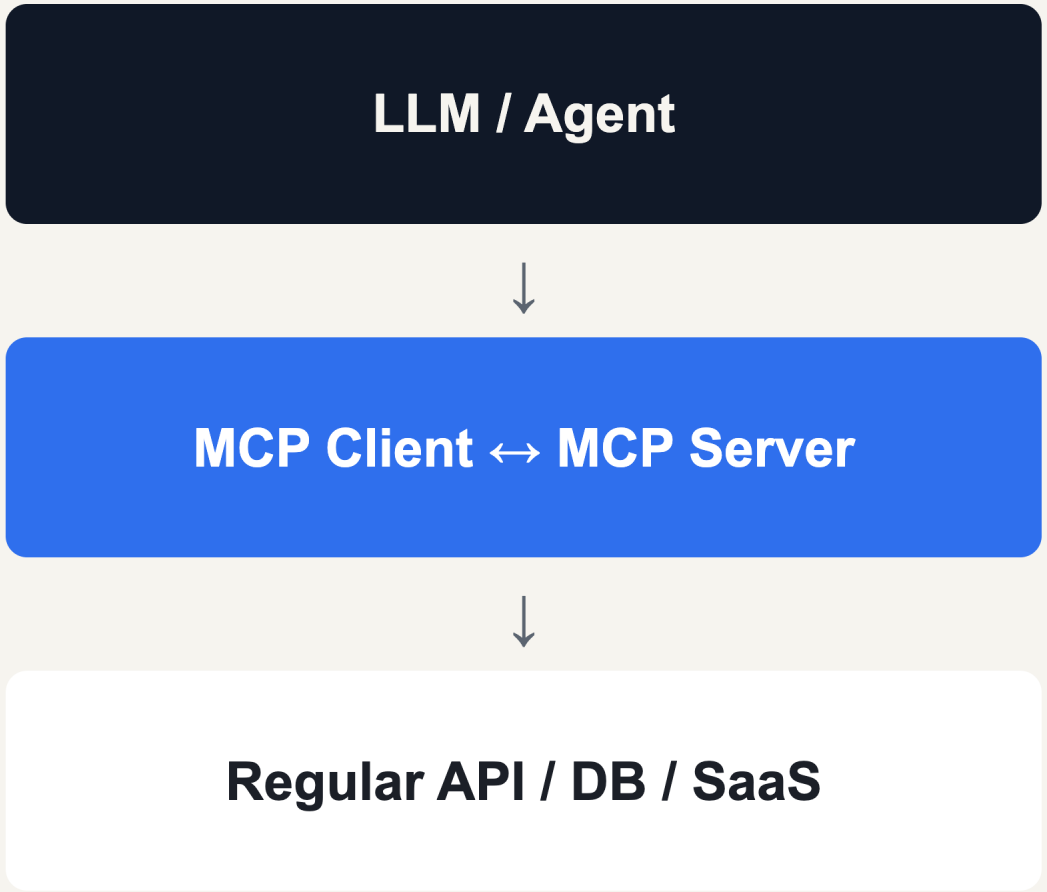
The result returns to the LLM for the final answer

Steps 3 to 5 repeat until the model has what it needs to answer.

MCP vs API at a Glance

Aspect	Traditional API	MCP
Consumer	Developer's code	AI model or agent via host
Integration	Custom per service	One uniform protocol
Discovery	Read docs, hard-code calls	Runtime: tools/list
Described for	Humans (OpenAPI)	The LLM (text + JSON schema)
State	Stateless request/response	Stateful, bidirectional session
Control flow	Your code picks each call	Model picks tool and timing
Best for	Pipelines, high-volume paths	Agents, multi-tool assistants

How MCP Layers on APIs



Function calling

How the model emits a structured tool call.

MCP

How tools are packaged, discovered and served in a standard way.

Together

The host converts MCP tool schemas into the model's tool-use format. MCP complements function calling and does not replace APIs.

Risks and Rule of Thumb

Risks to design for

- Prompt injection and tool poisoning via results or descriptions
- Over-permissioned servers: apply least privilege
- Context bloat from many tool schemas
- Auth and multi-tenancy for remote servers

Rule of thumb

- Quick agentic prototype or multi-tool assistant: use MCP
- Core logic, batch jobs, hot path: call the API directly
- Production often uses both

API is how software talks to a service. MCP is how AI agents discover and use it.